

A Verified CPU/GPU Solver for the 3D Heat Equation

Karl Keshavarzi

Department of Electrical and Computer Engineering

University of Waterloo

github.com/karl-kes/3d-heat-solver

Abstract—We present a C++/CUDA solver for the three-dimensional heat equation using an explicit (forward Euler) finite-difference scheme on a structured grid. The implementation supports two execution backends, an OpenMP/SIMD-vectorized CPU path and a CUDA GPU path, compiled from the same source tree and dispatched at compile time. We describe the numerical method, the memory layout and parallelization strategy for each backend, and a verification methodology that checks the simulation against a closed-form analytic solution, a conservation law, and an exact boundary-condition invariant, independently on both backends. This verification caught a genuine correctness defect in the original GPU boundary kernel, which we describe along with its fix. We report benchmark results from 8^3 to 512^3 grid cells, showing a crossover from CPU-favored to GPU-favored execution between 16^3 and 32^3 cells, and a speedup plateau around $65\text{--}68\times$ once both backends become memory-bandwidth bound.

I. INTRODUCTION

The heat equation is a standard model problem for explicit time-stepping finite-difference methods, and a common first project for learning GPU programming: the update rule is a simple, highly parallel stencil, but a correct implementation still requires getting boundary conditions, stability, and memory layout right on both a CPU and a GPU backend. This paper documents one such implementation, with an emphasis on verifying correctness rather than only reporting a speedup number. A simulation that runs fast but computes the wrong answer is not useful; we treat the performance results in Section V as meaningful only because Section IV independently establishes that both backends solve the stated equation correctly.

A. Related work

GPU finite-difference stencil computation is well studied; Micikevicius [1] describes shared-memory tiling and multi-GPU strategies for 3D finite-difference stencils on CUDA hardware, in the context of seismic modeling. The present work targets a single GPU and does not implement shared-memory tiling (Section VI), but the GPU kernel design follows the same global-memory stencil pattern that work uses as its un-tiled baseline.

II. MATHEMATICAL FORMULATION

A. Governing equation

We solve the unsteady heat equation on a cuboid domain,

$$\frac{\partial u}{\partial t} = \alpha \nabla^2 u, \quad (1)$$

where $u(x, y, z, t)$ is temperature and α is the (constant, isotropic) thermal diffusivity. The domain is discretized on a structured grid of $n_x \times n_y \times n_z$ interior cells with uniform spacings $\Delta x, \Delta y, \Delta z$, and the Laplacian is approximated with the standard second-order central-difference 7-point stencil,

$$\nabla^2 u_{i,j,k} \approx \frac{u_{i-1,j,k} - 2u_{i,j,k} + u_{i+1,j,k}}{\Delta x^2} + (\text{similarly in } y, z). \quad (2)$$

B. Time integration and stability

Time advancement uses explicit (forward) Euler,

$$u_{i,j,k}^{t+1} = u_{i,j,k}^t + \alpha \Delta t \nabla^2 u_{i,j,k}^t. \quad (3)$$

This scheme is conditionally stable; von Neumann stability analysis [3] of the 3D case gives the classical Courant–Friedrichs–Lewy (CFL) limit [2]

$$\Delta t \leq \frac{1}{2\alpha (\Delta x^{-2} + \Delta y^{-2} + \Delta z^{-2})}. \quad (4)$$

The implementation computes Δt from Eq. (4) with an 0.85 safety factor, so that changing the grid spacing or diffusivity from the command line cannot silently push the simulation into an unstable regime.

C. Boundary conditions

The domain boundary is insulated: zero heat flux normal to each face, $\partial u / \partial n = 0$. This is implemented with the standard ghost-cell mirroring technique, $u_{\text{ghost}} = u_{\text{adjacent interior}}$, applied independently on each of the six faces after every integration step. Because the central-difference stencil’s flux term at the boundary depends only on the difference between the ghost and adjacent-interior values, this mirroring makes that difference exactly zero, not merely approximately so.

III. IMPLEMENTATION

A. Dual-backend architecture

Both the CPU and GPU code paths live in the same `.cu/.cuh` source files, selected at compile time via `#if defined(__CUDAACC__)`. When no CUDA toolchain is available, CMake compiles the same sources as ordinary C++, and the project builds and runs with a pure CPU backend. This keeps the two implementations of each kernel textually adjacent, which made it straightforward to compare them directly when investigating the boundary-condition defect described in Section III-D1.

B. Memory layout

The field is stored in an aligned, struct-of-arrays container (`AlignedSoA`). Each dimension is padded up to a multiple of the SIMD width (16, 32, or 64 bytes, detected from `__AVX2__`/`__AVX512F__`) so that CPU stencil loops can be vectorized without unaligned loads, and so that the same padded layout is used unmodified on the GPU. Allocation is dispatched per backend: `cudaMalloc` under CUDA, `_aligned_malloc` on MSVC, and `std::aligned_alloc` elsewhere.

C. CPU parallelization

The CPU integration kernel parallelizes the two outer loop dimensions with `#pragma omp parallel for collapse(2)` and vectorizes the innermost loop with `#pragma omp simd` [5], relying on `__restrict__`-qualified pointers and an alignment hint (`__builtin_assume_aligned` on GCC/Clang, `__assume` on MSVC) so the compiler can generate aligned vector loads.

D. GPU kernel design

The integration kernel launches one thread per interior cell with a $8 \times 8 \times 8$ thread block [4], each thread computing one stencil update directly from global memory. The boundary condition is applied with three separate kernels, one per pair of opposing faces, each launched with a 2D thread grid sized to that face rather than to the full interior volume.

1) *A correctness defect found by testing:* The original boundary-condition kernel used a single 3D-launched kernel that derived all three face mirrors from one shared, interior-only-ranged thread index. Because that index never took the value 0 or $n - 1$ in any dimension, edge cells where two boundary faces meet (e.g. $i = 0, j = 0$, any interior k) never received their second face’s mirror update on the GPU, while the CPU implementation, which used a full range on the cross-axis in each face’s loop, did not have this defect. The discrepancy is invisible to a coarse correctness check: a center-value comparison never samples an edge, and a total-energy check is not sensitive to a thin line of stale cells. It was caught by an explicit per-face boundary test (Section IV) and fixed by splitting the single kernel into three per-face kernels with correctly-sized 2D launches, mirroring the CPU code’s loop ranges exactly. This also removed an efficiency problem in

the original kernel, which launched $O(n^3)$ threads to perform $O(n^2)$ worth of boundary work.

IV. VERIFICATION

Following standard code-verification practice [6], we check the implementation against problems with a known closed-form solution rather than relying on benchmark performance as a proxy for correctness. Because the CPU and GPU backends are selected at compile time, a single binary cannot run both, so direct cross-binary comparison is not a natural fit. Instead, the same test suite runs independently against the analytic solution in both build configurations; if both backends agree with the known-correct answer within tolerance, they are transitively in agreement with each other, without needing a dump-and-diff mechanism. Three checks are used, on a small (32^3) grid with an isotropic Gaussian initial condition.

A. Analytic solution check

The initial condition is an isotropic Gaussian, $u(\mathbf{r}, 0) = A_0 \exp(-r^2/2\sigma_0^2)$. For the heat equation on an effectively unbounded domain, this remains Gaussian for all t , with

$$\sigma^2(t) = \sigma_0^2 + 2\alpha t, \quad u(\mathbf{r}, t) = A_0 \left(\frac{\sigma_0^2}{\sigma^2(t)} \right)^{3/2} e^{-r^2/2\sigma^2(t)}. \quad (5)$$

The test runs the simulation for a fixed number of steps and compares the value at the grid point nearest the center against this closed form, within a 5% relative tolerance accounting for discretization error and the small residual influence of the (finite, insulated) domain boundary.

B. Conservation of energy

With zero flux at every boundary and no source term, $\int u dV$ is exactly conserved: $\frac{d}{dt} \int u dV = \alpha \oint \nabla u \cdot \mathbf{n} dA = 0$. The test compares the discrete sum $\sum_{i,j,k} u_{i,j,k}$ at $t = 0$ against the same sum after the run, within a 2% tolerance.

C. Exact boundary invariant

Because the boundary kernel implements $u_{\text{ghost}} = u_{\text{adjacent interior}}$ as a literal copy with no arithmetic, this is checked for *exact* equality (no tolerance) at every one of the six faces after the run. This is the test that caught the defect in Section III-D1.

D. Continuous integration

A GitHub Actions pipeline runs the full test suite against the CPU backend on every push, since hosted runners have no GPU available to execute CUDA kernels. The GPU backend is compiled in CI to catch build breakage (`nvcc` does not require a physical device to compile, only to run), but its tests are run manually against real hardware.

TABLE I
RUNTIME FOR 1000 FORWARD-EULER STEPS, AVERAGED OVER 5 OF 6
RUNS (FIRST DISCARDED AS WARM-UP).

Grid size	CPU (ms)	GPU (ms)	Speedup
8^3	4.14	32.2	$0.13\times$
16^3	13.8	22.7	$0.61\times$
32^3	83.7	26.7	$3.13\times$
64^3	535	31.8	$16.8\times$
128^3	3,805	62.0	$61.4\times$
256^3	29,856	446	$67.0\times$
512^3	225,321	3,336	$67.6\times$

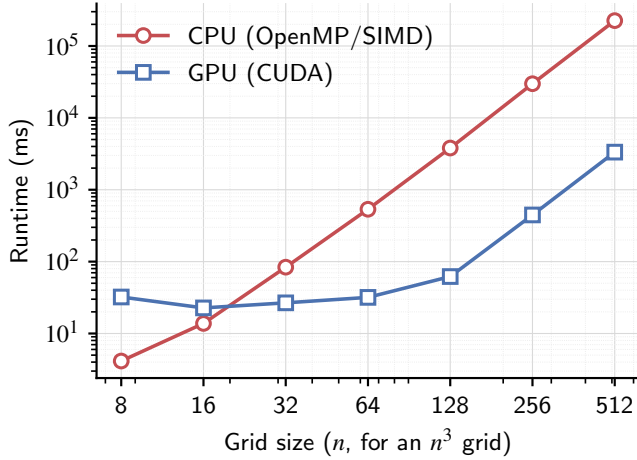


Fig. 1. Runtime vs. grid size for both backends (log-log).

V. PERFORMANCE EVALUATION

Table I and Figs. 1–2 summarize benchmark results measured on the development machine. At 8^3 and 16^3 cells, fixed CUDA kernel-launch overhead (four launches per step: one integration kernel and three boundary kernels) exceeds the actual compute, and the CPU backend is faster. The crossover occurs between 16^3 and 32^3 cells. Past the crossover, speedup grows quickly and then plateaus around $65\text{--}68\times$ from 128^3 cells onward, consistent with both backends becoming memory-bandwidth bound in the roofline sense [7]: the per-cell arithmetic intensity of a 7-point stencil is low (a handful of flops per two loads and one store), so runtime is dominated by reading and writing the field rather than by the floating-point operations themselves, and the GPU’s much higher memory bandwidth accounts for the plateau ratio.

VI. DISCUSSION AND LIMITATIONS

The solver targets a single GPU and a single explicit time-stepping scheme; it does not implement domain decomposition across multiple devices, shared-memory kernel tiling [1], or adaptive time stepping. Nor does it implement an implicit scheme such as Crank–Nicolson [8], which removes the CFL restriction at the cost of solving a linear system every step; the explicit scheme used here trades that restriction for a kernel with no inter-cell solve, which suits the goal

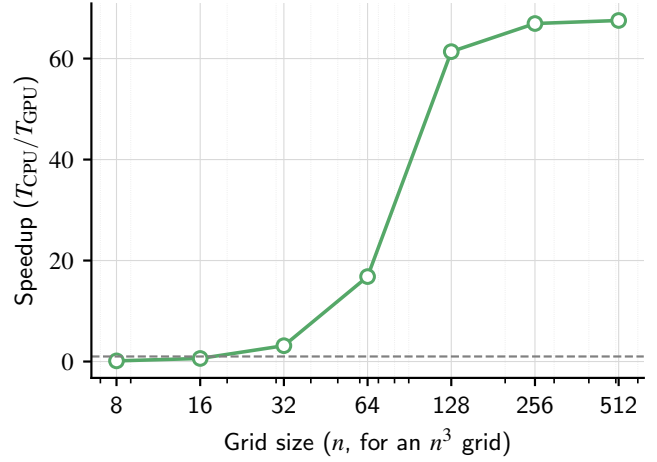


Fig. 2. Speedup (CPU time / GPU time) vs. grid size. The dashed line marks parity.

of a straightforward CPU/GPU comparison. The verification strategy, checking both backends independently against a closed-form solution rather than diffing them against each other, depends on having a problem with a known analytic solution; it would not directly generalize to a production case with complex geometry or material heterogeneity without a different reference (e.g. a manufactured solution or a trusted reference implementation).

VII. CONCLUSION

We have described and verified a dual-backend explicit finite-difference solver for the 3D heat equation. The verification methodology, checking both the CPU and GPU backends against a known analytic solution and an exact boundary invariant, found and allowed us to fix a real correctness defect in the original GPU boundary kernel that performance benchmarking alone did not reveal. With that fix in place, the GPU backend is correct on both checks and delivers a $65\text{--}68\times$ speedup over the OpenMP/SIMD CPU backend at grid sizes of 128^3 and above.

REFERENCES

- [1] P. Micikevicius, “3D finite difference computation on GPUs using CUDA,” in *Proceedings of the 2nd Workshop on General Purpose Processing on Graphics Processing Units (GPGPU-2)*, 2009.
- [2] R. Courant, K. Friedrichs, and H. Lewy, “Über die partiellen Differenzengleichungen der mathematischen Physik,” *Mathematische Annalen*, vol. 100, no. 1, 1928.
- [3] G. G. O’Brien, M. A. Hyman, and S. Kaplan, “A study of the numerical solution of partial differential equations,” *Journal of Mathematics and Physics*, vol. 29, no. 1, 1951.
- [4] NVIDIA Corporation, *CUDA C++ Programming Guide*, 2024.
- [5] L. Dagum and R. Menon, “OpenMP: an industry standard API for shared-memory programming,” *IEEE Computational Science and Engineering*, vol. 5, no. 1, 1998.
- [6] W. L. Oberkampf and T. G. Trucano, “Verification and validation in computational fluid dynamics,” *Progress in Aerospace Sciences*, vol. 38, no. 3, 2002.
- [7] S. Williams, A. Waterman, and D. Patterson, “Roofline: an insightful visual performance model for multicore architectures,” *Communications of the ACM*, vol. 52, no. 4, 2009.

- [8] J. Crank and P. Nicolson, "A practical method for numerical evaluation of solutions of partial differential equations of the heat-conduction type," *Mathematical Proceedings of the Cambridge Philosophical Society*, vol. 43, no. 1, 1947.